Roteiro de Gravação — M1S12 · Aula 1

Terça 13/10 · vídeo de 1h30 · Node.js, gerenciadores de pacotes e instalação global × local — roteiro completo, bloco a bloco

Desenvolvimento Front-End [React] T4 | Módulo 1 | Semana 12 | Professor: Lisandro Faria Pinheiro | **Versão de 08/10/2026.** Este documento é o **passo a passo para gravar** a aula de terça: o que preparar, como conduzir cada bloco, o que falar, o que aparece na tela, o que digitar e onde pedir pausa ao aluno. Os horários são os **minutos do vídeo final**. Siga junto com os slides 1–20 e com a pasta `Aula 1 - Terça 13-10`.

---

0. Visão geral da gravação

Item	Descrição
Vídeo	**Aula 1 — terça 13/10 — Node.js, npm e instalação global × local** · duração-alvo **1h30** (90 min)
Formato	Aula gravada, explicação técnica + **2 exercícios de live coding** (Ex1 e Ex2). A tarefa (Ex3) é feita pelo aluno depois.
Projeto	Quiz da Copa 2026 no terminal
Materiais	Slides 1–20 (PPTX revisado) · pasta `Aula 1 - Terça 13-10` (`Ex1` , `Ex2` , `Ex3` , cada um com `exercicio/` e `gabarito/`)
Estrutura	11 blocos (tabela abaixo). Os minutos são do **vídeo final**.

Linha do tempo

Minuto	Bloco	Slides	Tipo
00:00–00:03	1. Abertura e como usar o vídeo	1–2	Exposição
00:03–00:08	2. Checagem de ambiente e meta da semana	3–4	Demonstração
00:08–00:14	3. Node: o que é, REPL e Node × navegador	5–6	Demonstração
00:14–00:22	4. Gerenciadores de pacotes, npm e package.json	7–10	Exposição + demonstração
00:22–00:27	5. Versões (semver), lock e o npm install por dentro	11–12	Exposição dialogada
00:27–00:52	6. **Exercício 1 — Quiz da Copa com Node e npm**	13–15 (apoio)	Live coding (25 min)
00:52–00:59	7. Instalação local × global e quando usar cada	16–17	Exposição dialogada
00:59–01:15	8. **Exercício 2 — Global × local, devDependencies e npx**	18 (apoio)	Live coding (16 min)
01:15–01:20	9. Manutenção dos pacotes e segurança	19	Demonstração
01:20–01:26	10. Resumo da aula e a tarefa (Ex3)	20	Exposição

01:26–01:30	11. Encerramento e gancho para quinta	—	Exposição

1. Antes de gravar — preparação

O quê	Como fazer	✓
Local e áudio	Ambiente silencioso, microfone testado (grave 30 s e escute com fone), porta fechada, celular no silencioso. Fale a ~20 cm do microfone.	☐
Software de gravação	OBS Studio, Teams ou Camtasia: **1920×1080, 30 fps**, gravar tela inteira + microfone. Faça um teste de 1 minuto e confira se o áudio e o cursor aparecem.	☐
Tela limpa	Ative *Não perturbe* (Win + N), feche WhatsApp/e-mail/abas pessoais, esconda a barra de favoritos, deixe a área de trabalho vazia.	☐
VS Code legível	Zoom da interface (`Ctrl + =`) até a fonte do editor parecer ~20 px no vídeo; tema escuro de alto contraste; minimap desligado; fonte do terminal aumentada (Settings › `terminal.integrated.fontSize` : 18).	☐
Versões (seu ambiente)	Seu computador tem **Node v20.18.0 e npm 10.8.2** e **não pode ser alterado** (regra da empresa). Grave com ele: `node --version` mostrará `v20.18.0` e o slide 3 mostra 24.x — diga *"a versão que aparece pode ser outra; o importante é aparecer um número"*. Use `PROJETO DO PROFESSOR (Node 20.18.0)/etapa-1-npm-commonjs` como consulta (já instalado, testado e comentado).	☐
Internet e cache	Rode **uma vez, antes de gravar**, os `npm install` do exercício em uma pasta de teste: o cache do npm deixa as instalações quase instantâneas no vídeo (e evita gravar tela de espera). Mesmo assim, grave com internet ligada.	☐
Pastas prontas	Copie as pastas `exercicio` para um local limpo (Ex.: `C:\gravacao\`) **sem** `node_modules` e sem `package.json` (quando o roteiro disser que será criado). Deixe o `gabarito` aberto em outra janela do VS Code, **fora do enquadramento**, para consulta.	☐
Slides	Abra o PPTX revisado em modo apresentador; o slide de cada bloco está indicado no roteiro. Alterne com `Alt + Tab` .	☐
Terminal	Use o **Prompt de Comando** ou o **Git Bash** no VS Code (evita a trava `npm.ps1` do PowerShell). Limpe o terminal com `cls` a cada bloco.	☐

Específico da terça

Antes de gravar o Bloco 8, **desinstale o nodemon global** se ele já existir (`npm uninstall -g nodemon`) para a demonstração partir do zero. Seu Node 20.18.0 e seu npm 10.8.2 funcionam normalmente com tudo da terça (`prompt-sync` , `chalk@4` , `nodemon` 3.1.x e `npx`): só os números de versão exibidos diferem dos alunos. Se o laboratório não permitir `-g` (erro `EACCES`), grave o passo global só mostrando `npm ls -g --depth=0` e explicando — o aprendizado se mantém.

2. Como conduzir a gravação

- **Grave em blocos.** Cada bloco do roteiro é um "take" independente (de 3 a 25 min). Se errar feio, regrave só o bloco — não o vídeo inteiro. Na edição você junta os blocos.
- **Abra cada bloco com uma frase-âncora** (ex.: *"Bloco 6: vamos construir o Quiz da Copa"*) e diga o minuto-alvo; isso ajuda a localizar o corte e a criar os capítulos.
- **Diga o que digita.** Quem assiste não vê seu teclado: fale *"npm espaço install espaço prompt-sync"* na primeira vez e depois só o comando.
- **Espere 2 segundos** depois de cada comando antes de falar do resultado; deixe a saída aparecer inteira na tela.
- **Cursor como ponteiro:** circule/aponte o que está explicando. Dê zoom (`Ctrl + =` no editor) em trechos pequenos.
- **Pausas pedagógicas ⏸:** nos pontos marcados, diga *"pause o vídeo e faça"*, mostre na tela o cartão **⏸ SUA VEZ** e deixe ~5 s de silêncio (você corta e ajusta na edição).
- **Se errar um comando:** não finja que nada aconteceu — erros reais são didáticos. Se for só tropeço de fala, pare 3 s, diga *"vou repetir"* e refaça a frase; corte na edição.
- **Não leia os slides.** O slide é o apoio; a fala do roteiro é uma *referência natural*, não um texto para decorar.
- **Ritmo:** 140–150 palavras por minuto. Se um bloco estourar, corte a demonstração opcional (marcada como ⭐ no roteiro), nunca o exercício guiado.

3. Roteiro bloco a bloco

00:00–00:03 Abertura e como usar o vídeo (slides 1–2)

Na tela

Slide 1 (título) → slide 2 (agenda).

O que falar:

"Olá, turma! Bem-vindos à Semana 12 do Módulo 1. Esta aula é **gravada**: eu vou programar, e vocês programam junto. Sempre que aparecer o aviso ⏸ **Sua vez**, pausem o vídeo, façam o passo e só depois continuem. Esta semana o JavaScript sai do navegador: hoje **Node e npm**; na quinta, ao vivo, **módulos**; e na sexta, em vídeo, o **debugger**. O fio condutor é um **Quiz da Copa** que roda no terminal. Vamos lá?"

Cuidado

Mostre a agenda do slide 2 apontando: terça = vídeo, quinta = ao vivo, sexta = vídeo.

00:03–00:08 Checagem de ambiente e meta da semana (slides 3–4)

Na tela

Slide 3 → alterne para o terminal do VS Code → slide 4.

O que falar:

"Antes de qualquer coisa: o Node responde? Abram o terminal e rodem **node --version** e **npm --version**. Apareceu um número? Então vocês já têm Node e npm — o npm vem junto. Se aparecer 'não é reconhecido', fechem e reabram o terminal ou instalem a versão LTS: **um comando só**, `winget install OpenJS.NodeJS.LTS` (o passo a passo com fluxograma está no Guia de Instalação — mostre a imagem na tela por 10 segundos e diga que o arquivo está no AVA). Se o PowerShell reclamar de **npm.ps1**, não é falta de administrador: é a trava de scripts; usem o Prompt de Comando ou o Git Bash."

```
node --version
npm --version
```

⏸ Pausa para o aluno

Peça: *"pause e rode os dois comandos; me diga a versão que apareceu"*.

Slide 4:

"A meta da semana: um Quiz da Copa de terminal. Ele nasce hoje, vira módulos na quinta e é depurado na sexta. E vamos jogar Kahoot em aula também."

00:08–00:14 Node: o que é, REPL e Node × navegador (slides 5–6)

Na tela

Slides 5 e 6; depois o terminal.

O que falar:

"Node.js é um **ambiente de execução**: roda JavaScript fora do navegador, direto no computador. É a mesma linguagem das semanas anteriores; muda o palco: em vez de página, o terminal. E por que importa? O React do Módulo 2 roda sobre Node — *npm create vite*, *npm install*, *npm run build* são todos Node."

```
node
> 2 + 2
> typeof window
> typeof process
> .exit

node -e "console.log(2 + 2)"
```

Ao digitar:

"Reparem: **typeof window** dá *undefined* — no Node não existe janela nem DOM. Já **typeof process** dá *object*: o Node tem recursos próprios. O modo interativo, o REPL, serve para testes rápidos. O trabalho de verdade fica em arquivos .js."

> **Pergunta:** O que o Node tem que o navegador não tem, e vice-versa? **Resposta esperada:** Node: `process`, `fs`, `require`; navegador: `window`, `document`, DOM, `localStorage`.

00:14–00:22 Gerenciadores de pacotes, npm e package.json (slides 7–10)

Na tela

Slides 7, 8, 9 e 10. Em 1 minuto, abra **npmjs.com** e pesquise *chalk* (mostre downloads semanais, versões e o repositório).

O que falar:

"Um **pacote** é código pronto que alguém publicou. Um **gerenciador de pacotes** busca, baixa, guarda a versão certa e resolve as dependências das dependências. O npm é o padrão e já vem com o Node; existem o yarn e o pnpm — fazem o mesmo trabalho, mudam o comando e o arquivo de lock. Regra de ouro: **um** gerenciador por projeto. O coração do projeto é o **package.json**: nome, versão, scripts e dependências. Ele nasce com **npm init -y**. Duas seções importam: **dependencies** — o que o código precisa para rodar — e **devDependencies** — o que só usamos para desenvolver."

> **Pergunta:** Por que não basta copiar os arquivos da biblioteca para a pasta? **Resposta esperada:** Sem versão registrada, sem as dependências dela, cada máquina fica diferente — o "na minha máquina funciona".

Segurança

Diga em voz alta: confira nome, downloads e mantenedor antes de instalar; a IA às vezes inventa nomes de pacote.

00:22–00:27 Versões (semver), lock e npm install por dentro (slides 11–12)

Na tela

Slides 11 e 12.

O que falar:

"Versão tem três números: MAIOR, MENOR, CORREÇÃO. O circunflexo em **^4.1.2** aceita qualquer 4.x acima, nunca o 5; o til aceita só 4.1.x. O **package-lock.json** congela as versões exatas de tudo o que foi instalado — ele vai para o Git. A pasta **node_modules** é pesada e recriável: não vai para o Git, vai no .gitignore, e é refeita com **npm install** sem nome. O npm install de um pacote faz quatro coisas: consulta o registro, baixa para node_modules, anota no package.json e grava o lock."

Pergunta: Se eu apagar a node_modules, perco o projeto? **Resposta esperada:** Não: `npm install` recria a pasta a partir do `package.json` + lock.

00:27–00:52 EXERCÍCIO 1 — Quiz da Copa com Node e npm (live coding) (slides 13–15 apoio)

Pasta

`Aula 1 - Terça 13-10/Ex1 - Quiz da Copa com Node e npm (guiado)/exercicio` aberta no VS Code. **Gabarito de consulta:** `.../gabarito/index.js` (comentado linha por linha). Slides 13–15 servem de apoio ao digitar (`node arquivo.js` , `npm install chalk@4` , cores com chalk).

Min	Passo	O que fazer / digitar
00:27	Mostrar a pasta	Abrir o `index.js` : **as 10 perguntas já estão prontas** (dados) e há `TODO` onde vamos escrever. Mostrar o `.gitignore` .
00:29	Criar o package.json	`npm init -y` → abrir o arquivo e comentar `name` , `version` , `main` , `scripts` .
00:31	Primeiro pacote	`npm install prompt-sync` → mostrar: `dependencies` no package.json, a pasta `node_modules` e o `package-lock.json` aparecendo.
00:34	Importar o prompt	Digitar no `index.js` o `require` do prompt-sync (ver código abaixo).
00:35	Regras	Digitar `verificar` e `mensagemFinal` . **Explicar o `Number(...)` :** o prompt devolve texto.
00:38	Fluxo	Título, `acertos` , laço `for` , mostrar pergunta e opções, `prompt` , `if/else` , mensagem final.
00:44	Rodar	`node index.js` → responder 2 perguntas (uma certa, uma errada). ⏸ **Sua vez**.
00:45	Cores	`npm install chalk@4` → `require("chalk")` → pintar título, ✅ verde e ❌ vermelho. Explicar por que a **versão 4** (a 5+ é ESM — pista para quinta).

00:48	Script start	No package.json: `"start": "node index.js"` → `npm start`.
00:50	Teste do clone	Apagar a pasta `node_modules` (Explorer › Delete) → `npm start` falha com `Cannot find module` → `npm install` → `npm start` funciona. ⏸ **Sua vez.**
00:51	⭐ Bônus (1 min)	Mostrar que a pasta `web/` (HTML + CSS + JS) já está pronta: acrescentar o script `"web": "npx -y http-server web -p 5500 -c-1"` → `npm run web` → abrir `http://127.0.0.1:5500`. "O mesmo quiz, no navegador; na quinta vocês veem por que ele usa `import`." Não digite o HTML/CSS.

Código que você digita (versão enxuta; o comentado está no gabarito)

```
const promptSync = require("prompt-sync");
const prompt = promptSync();
const chalk = require("chalk");          // depois do npm install chalk@4

function verificar(resposta, correta) {
  return Number(resposta) === correta;
}

function mensagemFinal(acertos, total) {
  return `Você acertou ${acertos} de ${total}! 🏆`;
}

console.log(chalk.bold.yellow("=== QUIZ DA COPA 2026 ==="));
let acertos = 0;

for (let i = 0; i < perguntas.length; i++) {
  const q = perguntas[i];
  console.log(chalk.cyan(`\n${i + 1}) ${q.pergunta}`));
  q.opcoes.forEach(function (opcao, indice) {
    console.log(`   ${indice + 1}. ${opcao}`);
  });
  const resposta = prompt("Sua resposta (1-4): ");
  if (verificar(resposta, q.correta)) {
    console.log(chalk.green("✅ Acertou!"));
    acertos++;
  } else {
    console.log(chalk.red(`❌ Errou! A resposta certa era a opção ${q.correta}.`));
  }
}

console.log(chalk.bold.yellow("\n" + mensagemFinal(acertos, perguntas.length)));
```

Fechamento do exercício:

"Pronto: um programa Node de verdade, com dependências, script e que qualquer pessoa reconstrói com **npm install**. Repare no que viajou com o projeto: o package.json e o lock — a node_modules, não."

> ⏸ **Pausa para o aluno**
>
> ⏸ **Sua vez** no minuto 00:44 (rodar) e 00:50 (teste do clone). Mostre na tela o cartão e deixe ~5 s.

Se der errado

Quiz sempre erra → faltou `Number()`. **Cannot find module 'prompt-sync'** → esqueceu o `npm install` ou está na pasta errada (`pwd`). **Texto sem cor** → esqueceu o `require("chalk")` . **PowerShell bloqueando** → troque para cmd/Git Bash.

00:52–00:59 Instalação local × global e quando usar cada (slides 16–17)

Na tela

Slides 16 e 17; terminal com a pasta do Ex1.

```
npm ls --depth=0
npm ls -g --depth=0
npm root -g
```

O que falar:

"Até agora tudo o que instalamos foi **local**: vai para a node_modules do projeto e fica anotado no package.json. Existe também o **global**, com **-g**: instala uma ferramenta de linha de comando para a máquina inteira, mas **não aparece no package.json** — não viaja com o projeto. O sintoma clássico: funciona no seu computador e quebra no do colega. Regra de bolso: o que o **código importa** é local, em dependencies; ferramenta de **desenvolvimento** é local com **-D**; comando de **uso único** é npx; global só para utilitário pessoal."

Pergunta: O `chalk` pode ser instalado com `-g` para o meu `require` achá-lo? **Resposta esperada:** Não: o `require` procura na `node_modules` do projeto; global serve para comandos, não para bibliotecas.

00:59–01:15 EXERCÍCIO 2 — Global × local, devDependencies e npx (live coding) (slides 18 apoio)

Pasta

`Aula 1 - Terça 13-10/Ex2 - Global x Local, devDependencies e npx (guiado)/exercicio` (arquivo `saudacao.js` pronto + `ROTEIRO-COMANDOS.md`). Siga a tabela de comandos do roteiro; abaixo, só a narrativa.

Min	Passo	Comando / fala
00:59	Olhar o global	`npm ls -g --depth=0` , `npm root -g` , `npm config get prefix` — "é aqui que o global mora".
01:01	Instalar global	`npm install -g nodemon` → `nodemon saudacao.js` ; editar o texto de `saudacao.js` e salvar → **[nodemon] restarting**. `Ctrl + C` .
01:05	Mostrar o problema	`npm init -y` → abrir o package.json: **o nodemon não está lá**. "Se eu mandar isto ao colega, ele não tem a ferramenta."
01:07	Instalar local (-D)	`npm install --save-dev nodemon` → mostrar `devDependencies` no package.json.
01:09	Script dev	Em `scripts` : `"dev": "nodemon saudacao.js"` → `npm run dev` → editar e salvar de novo.
01:12	npx	`npx nodemon --version` (usa o local); `npx cowsay "Copa 2026"` (roda uma vez, sem deixar rastro; mostre que o package.json não mudou).

01:14	Limpar	`npm uninstall -g nodemon` .	⏸ **Sua vez:** preencher a tabela "Regra de bolso" do LEIAME.

> **Se der errado**
>
> **EACCES** no `-g` → não use administrador; pule o passo global e narre. **'nodemon' não é reconhecido** logo após o `-g` → feche e reabra o terminal (PATH). **cowsay** sem internet → mostre só o comando e explique.

Fechamento do exercício:

"O que ficou: global = ferramenta pessoal, invisível para o projeto; local com -D = ferramenta de desenvolvimento registrada para todo o time; npx = rode sem instalar."

01:15–01:20 Manutenção dos pacotes e segurança (slides 19)

Na tela

Slide 19; terminal na pasta do **Ex1/exercicio** (com chalk@4).

```
npm outdated
npm audit
```

O que falar:

"**npm outdated** mostra o que está desatualizado. Olhem o chalk: o Current é 4.1.2 e o **Latest é bem mais novo** — a versão MAIOR mudou. Mudança de MAIOR pode quebrar o projeto. É exatamente o que vamos provocar de propósito na quinta. O **npm audit** procura vulnerabilidades conhecidas. E lembrem: confira o nome do pacote antes de instalar — nomes parecidos podem ser armadilha, e a IA às vezes inventa pacotes que não existem."

> ⏸ **Pausa para o aluno**
>
> Peça que pesquisem *chalk* no npmjs.com e anotem a versão mais recente (será usada na quinta).

01:20–01:26 Resumo da aula e a tarefa (Ex3) (slides 20)

Na tela

Slide 20; depois abra a pasta `Ex3 - Tarefa`.

O que falar:

"Resumindo: Node executa JavaScript fora do navegador; o npm instala pacotes e roda scripts; o package.json é a identidade do projeto; dependencies versus devDependencies; local versus global; npx para rodar sem instalar. A tarefa é o **Ex3**: ampliar o Quiz com mais cinco perguntas, calcular o percentual, classificar o jogador — Craque, Titular, Reserva ou Torcedor — e, como estrela, instalar o nodemon como devDependency. O LEIAME da pasta explica cada passo. Entreguem conforme combinado."

01:26–01:30 Encerramento e gancho para quinta (sem slide)

O que falar:

"Na quinta estamos juntos, **ao vivo**, 19h. A aula abre com um Kahoot sobre o que vimos hoje. E um aviso: o index.js do nosso Quiz cresceu e, na quinta, vamos separar em módulos — e vamos atualizar o chalk para a versão mais nova e ver o que acontece. Quem quiser chegar na frente: tenha o Ex1 funcionando com **npm start**. Não esqueçam a Atividade Avaliativa no AVA. Até quinta!"

> **Cuidado**
>
> Termine olhando para a câmera/tela com calma; deixe 5 s de tela final para o corte.

4. Plano B e pontos de atenção

- **Sem internet na hora de gravar:** aqueça o cache do npm antes e use `npm install --prefer-offline`; se não houver jeito, use a pasta `gabarito` (traz `package-lock.json`) e explique o que o comando faria.
- **Erro inesperado ao vivo:** leia a mensagem inteira em voz alta — é conteúdo da semana. Só corte se o erro for de ambiente (e não de conceito).
- **Seu Node é o 20.18.0:** o `chalk@4` funciona em qualquer versão; a diferença só aparece na quinta (`ERR_REQUIRE_ESM` no seu Node × `TypeError` nos alunos), e o guia ensina a reproduzir o erro dos alunos com `node --experimental-require-module`.
- **Tempo estourando:** encurte o Bloco 4 (conceitos de gerenciadores) e o Bloco 9; nunca o teste do clone nem o nodemon local × global.

5. Depois de gravar — edição e publicação

- Corte silêncios longos, tropeços e esperas de instalação (mantenha a saída do comando visível por 2–3 s).
- Acrescente o cartão ⏸ **SUA VEZ** nos pontos de pausa e, se possível, legendas automáticas (revise nomes de comandos).
- Nome do arquivo: `M1S12 - Aula 1 - Terca 13-10 - Node e npm.mp4` — duração final entre **1h25 e 1h35**.
- Publique no AVA com os **capítulos** abaixo na descrição e anexe: PDF dos slides, a pasta de exercícios (ZIP **sem** `node_modules`) e, **depois do prazo**, os gabaritos.
- Avise a turma (e a monitoria) com o link, a duração e a tarefa; lembre da Atividade Avaliativa Semanal.

Capítulos sugeridos (copie para a descrição do vídeo)

Minuto	Capítulo
00:00	Abertura e como usar o vídeo
00:03	Checagem de ambiente: node e npm
00:08	O que é Node (REPL, Node × navegador)
00:14	Gerenciadores de pacotes, npm e package.json
00:22	Versões (semver), lock e npm install
00:27	Exercício 1 — Quiz da Copa com Node e npm
00:52	Instalação local × global
00:59	Exercício 2 — nodemon global × local, devDependencies e npx
01:15	Manutenção dos pacotes e segurança
01:20	Resumo e tarefa (Ex3)
01:26	Encerramento